

complete study notes

Generative AI & LangChain

everything you need to know — explained simply

Documents → Chunks → Embeddings → Vector Store → Retriever → Context → LLM → Answer

1 What is Generative AI?

AI that **creates new content** — text, images, music, video, code, speech — based on patterns learned from data. (e.g. ChatGPT, image generators, code tools)

Ex: "Write a story about a robot on Mars" → AI creates a brand-new story, doesn't just retrieve one.

Traditional AI vs Generative AI

Traditional AI	Generative AI
Classifies / predicts	Creates new content
Spam detection → "Spam"	Writes a full email
Photo → "Dog" (label)	"Draw a dog on a beach" → new image



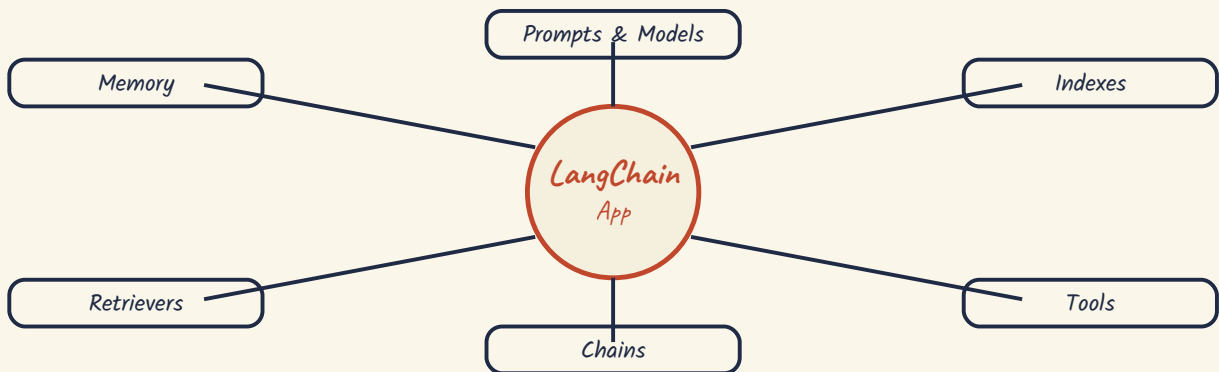
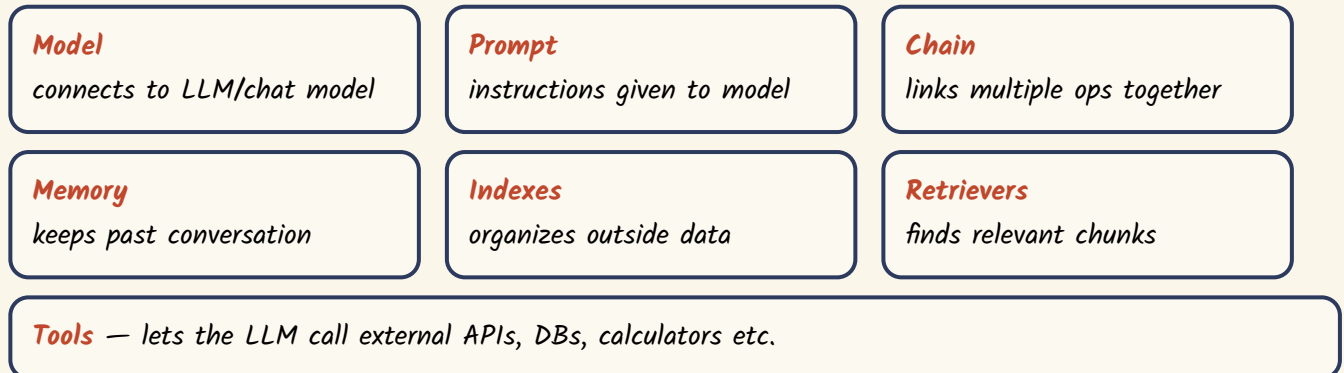
Remember: Traditional → understands/predicts · Generative → creates

2 What is LangChain?

A **framework for building LLM-powered apps**. An LLM alone just generates text — real apps need prompts, memory, retrieval, tools too. LangChain connects all these steps.

PDF → Extract text → Split → Embed → Store → Search → Build prompt → Send to LLM → Answer

Core Components



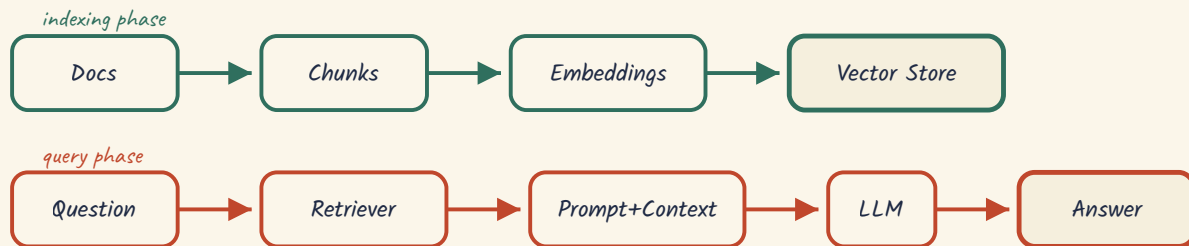
Why LangChain? (advantages)

- **Modular** — combine components easily
- **Reusable** — prompts/retrievers reused across apps
- **Integration** — connects to models, DBs, vector stores, APIs
- **RAG support** — makes retrieval-based apps easier to build
- **Memory** — keeps conversational context
- **Chains** — links multiple steps into a workflow

3 Retrieval-Augmented Generation (RAG)

LLM **retrieves relevant info** from an outside source before answering — instead of relying only on training data.

Why? An LLM doesn't know your private docs (Company Policy.pdf, HR Guidelines.pdf...). RAG searches them and feeds the answer-relevant bits to the model.



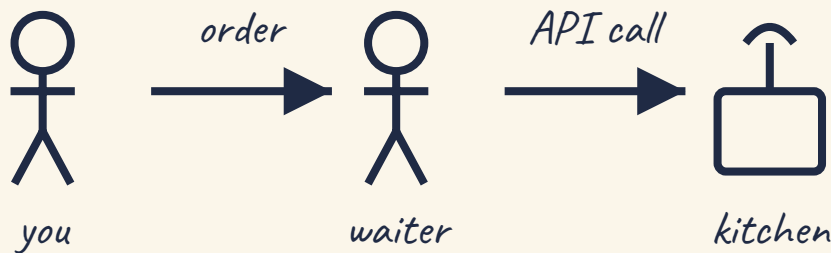
Step-by-step

1. **Load** docs (PDF, Word, web, DB, text)
2. **Split** into chunks (large docs → smaller pieces)
3. **Embed** each chunk → numeric vector
4. **Store** vectors in a vector DB (FAISS, Chroma, Pinecone, Weaviate)
5. User asks a **question** → embedded too
6. **Retrieve** semantically similar chunks
7. **Augment** — add retrieved text into the prompt
8. **Generate** — LLM answers using that context

Remember: RAG = retrieve + give to LLM + generate answer

4 What is an API?

Lets two apps **communicate**. Think of it as a **waiter** — you never enter the kitchen yourself.



You → waiter(API) → kitchen(server) → waiter → you

Request-Response cycle

Client sends request → API receives → server processes (queries DB) → server builds response → API returns it to client.

```
GET /users/123  
→ { "id": 123, "name": "Rahim" }
```

Key terms

- **Request** — what client asks for
- **Response** — what server sends back
- **Endpoint** — a specific API URL/function
- **HTTP methods:** GET → read · POST → create · PUT → update · DELETE → remove

5 What is an API Key?

A **credential** that authenticates your app to the API — "who's making this request, and are they allowed?"

Used to: authenticate requests · link usage to an account · apply permissions · measure billing · block unauthorized use.

Generate & use one (OpenAI example)

1. Create key in your developer dashboard
2. Store as an environment variable — never hardcode it
3. Install the SDK: `pip install openai`
4. Read it in code via `os.getenv()`

```
export OPENAI_API_KEY="your_key_here"

import os
api_key = os.getenv("OPENAI_API_KEY") (safe)
api_key = "sk-xxxxxxx" (never do this)
```

6 Model Component — LLM vs Chat Model

LLM	Chat Model
Text completion	Built around conversations
Input: plain text prompt	Input: system/user/assistant messages
Output: text completion	Output: an AI message

LLM: Text → Model → Text / Chat Model: Messages → Model → AI Message

Modern APIs mostly use the message-based interface either way — it's an abstraction difference, not a hard capability limit.

7 Prompt & PromptTemplate

A **prompt** = the instruction given to the model. Good prompts specify: what to do, what info to use, style, and format.

A **PromptTemplate** is a reusable prompt with placeholders, so you don't rewrite it each time.

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
    "Explain {topic} in simple words."
)
prompt.format(topic="Generative AI")
→ "Explain Generative AI in simple words."
```

Why use it: reusability · consistency · dynamic input (e.g. `{topic}` + `{audience}`)

8 Chain Component

A **chain** connects multiple steps into one workflow — output of one step feeds the next.

Simple: User Input → Prompt → LLM → Output

RAG-style: Question → Retriever → Docs → Prompt → LLM → Answer

Similar to an **ML pipeline**: Raw Data → Preprocessing → Feature Engineering → Model → Prediction — each stage passes its output forward.

9 Indexes — the RAG data pipeline

Documents → Loader → Splitter → Embedding Model → Vector Store → Retriever

1 Document Loaders

Load data from PDFs, text files, websites, CSVs, Word docs, databases into a usable format.

2 Text Splitters

Break large docs into smaller chunks (~500–1000 chars, with overlap). Why: easier retrieval, better accuracy, smaller context per call.

3 Embedding Models

Convert text → numeric vectors capturing meaning. Similar-meaning texts → similar vectors.

```
"Python is a programming language."  
→ [0.12, -0.45, 0.78, ...]
```

4 Vector Stores

Store embeddings + run similarity search. Examples: FAISS, Chroma, Pinecone, Weaviate.

5 Retrievers

Search the vector store and return the most relevant chunks for a query.

Ex: "vacation policy?" → Chunk 17 (vacation policy), Chunk 42 (annual leave), Chunk 51 (benefits)

10 Memory Component

Lets the app remember earlier turns in a conversation.

Without memory: "My name is Rahim" → ... → "What's my name?" → "I don't know."

With memory: → "Your name is Rahim." (correct)

1 ConversationBufferMemory

Stores the **entire** conversation. + full context – grows large over time.

2 ConversationBufferWindowMemory

Keeps only the **last N** messages, drops older ones. + smaller context – forgets old info.

3 ConversationSummaryMemory

Stores a **running summary** instead of full text. + saves space – details can get lost.

4 Custom Memory

Developer-designed – e.g. store customer ID, preferences, past issues. + full control – more work to build.

Type	Stores	Trade-off
Buffer	whole chat	grows large
Buffer Window	last N turns	forgets old context
Summary	running summary	can lose detail
Custom	developer-defined	more effort to build

The whole picture, memorized

Data side: Load → Split → Embed → Store → Retrieve

LLM side: Prompt → Model → Answer

App side: Chain + Memory + Tools/Retrievers

#GenerativeAI

#LangChain

#RAG

#LLM

#API

#AIEngineering

– keep learning, one chunk at a time